CYBER RANGE

ENGINEERING DOCUMENTATION PACK

Blackstone Auxiliary Field
Engineering Reference

Security Onion & Splunk military-airfield range

Architecture, build pipeline, telemetry coverage, verification and operations for the **blackstone.mil** range: a two-domain base enterprise with a flight-ops child domain, a layer-3–isolated fuel-farm OT enclave, a bespoke air-traffic-control subsystem, and the distributed Security Onion grid — with an optional Splunk cluster and Malcolm OT appliance — that watches all of it.

Security Onion 2.4 | 4 sensors | Fuel-farm OT | ATC subsystem | Splunk cluster (stacked) | Malcolm 26.08.0

AUDIENCE

Engineers who build, extend, operate and debug the range. Assumes working knowledge of Security Onion, Active Directory, Ansible, and the SimSpace platform.

SCOPE

The range as deployed by its Ansible overlay. Every variant runs a distributed Security Onion grid; one variant additionally runs a distributed Splunk cluster, with a universal forwarder on every managed host, plus a Malcolm OT appliance. Generated 2026-11-03.

Contents

Introduction 4

01 Architecture 6

02 Network 9

03 Active Directory 13

04 Inventory and Roles 15

05 Build and Deploy 20

06 Security Monitoring and Telemetry 24

07 Verification 28

08 Operations and Troubleshooting 31

Blackstone Auxiliary Field — Engineering Documentation Pack

Engineering reference for the **Blackstone Auxiliary Field** cyber range: a two-domain military-base enterprise across `blackstone.mil` and its flight-ops child `fops.blackstone.mil`, a layer-3–isolated fuel-farm OT enclave, a bespoke air-traffic-control subsystem, and the blue-team instrumentation that watches all of it. This pack is for engineers who **build, extend, operate and debug** the range — not for trainees or event coordinators.

Audience. Engineers who build, extend, operate and debug the range. Assumes working knowledge of Security Onion, Active Directory, Ansible, and the SimSpace platform.

Scope. The range as deployed by its Ansible overlay. Every variant runs a distributed Security Onion grid as its network-security monitoring; one variant additionally runs a distributed Splunk cluster, with a universal forwarder on every managed host feeding it alongside the Elastic Agent, plus a Malcolm appliance for OT deep-packet analysis. Where behaviour differs between the monitoring-only and the Splunk-stacked variants, the text calls it out.

For the operator view see the **Range Guide**; for the one-page capability summary see the **Range Overview**. This pack goes deeper into design rationale, the build/verify pipeline, the inventory and role model, and the failure modes you will actually hit.

Contents

CHAPTER	COVERS
01 · Architecture	Purpose, design decisions, system architecture, data/telemetry flows, segmentation model
02 · Network	Addressing plan, transit links, the OSPF/eBGP/static routing model, firewalls, DNS, the GRE mirror fabric
03 · Active Directory	Two-domain forest, four DCs, parent/child trust, members, emulated users, DNS/mgmt hazards
04 · Inventory & roles	Inventory groups and their meaning, host→group mapping, custom + base roles, group_vars hierarchy
05 · Build & deploy	Tarball build, build-time gates, blueprint fetch + deploy, the 3-attempt retry model, phase order, vault, Nexus artifact sourcing
06 · Security monitoring & telemetry	The Security Onion grid, sensor placement, registry seeding, Fleet integrations, endpoint telemetry, the Splunk cluster + universal forwarders, Malcolm OT DPI
07 · Verification	Build-time verifiers, runtime health checks, what "healthy" looks like
08 · Operations & troubleshooting	Known behaviours, common pitfalls, secrets/rotation, re-deploy/reset, blueprint re-pinning

Authority order

When sources disagree, trust them in this order:

1. **The platform blueprint** — the definitive list of what VMs exist, their images, IPs and sizing.
2. **The inventory** — group membership, which drives what Ansible does to each host.
3. `group_vars` / `host_vars` — the single source of truth for addressing, credentials, pins and tunables.
4. **This pack** — narrative and rationale over the above.

A number in prose is only as good as the blueprint and inventory it was derived from. Where this pack states a count (host totals, Windows totals, sensor coverage, …) it was computed from the range's current blueprint and inventory; re-derive after any change rather than trusting the text.

01 — Architecture

What this range is

Blackstone Auxiliary Field is a simulated military airfield built as a **defensive cyber range** on the SimSpace NG platform. It exists to train and assess SOC analysts against a network that behaves like a real base IT/OT estate: a two-domain Active Directory forest, a DMZ, a segmented fuel-farm OT enclave running real ICS equipment, a bespoke air-traffic-control subsystem fed from recorded flight data, continuous emulated-user noise, and a distributed Security Onion grid watching from four vantage points. One variant additionally runs a distributed Splunk cluster, with a universal forwarder on every managed host feeding it alongside the Elastic Agent, plus a Malcolm appliance for OT deep-packet analysis.

The whole range is **Ansible from a pinned tarball**. It stands up, re-deploys and resets idempotently so an exercise can be run, torn down and rebuilt clean.

The range is built by a **range-specific overlay** on top of SimSpace's core Ansible library. The overlay ships inventory, `host_vars`, `group_vars`, custom roles and range playbooks, then bundles the base roles it uses plus its own overlays into the deploy tarball.

Design decisions that shape everything downstream

These five properties are load-bearing — most behaviours only make sense against them.

1. **Sensor coverage is positional.** Each Security Onion sensor receives a layer-2 mirror from exactly one router. An event is visible only to the sensor whose router carries it; there is no aggregation tap. Correlating an intrusion across segments means correlating across sensors, and which sensor carries an event is itself a location signal. See the security monitoring & telemetry chapter.
2. **The OT enclave is segmented at layer 3.** `bs-ops-fw` is a default-deny boundary between the base enterprise and the OT chain, and `bs-modbus-gateway` is the single routed path into the fuel-farm Purdue segments. OT reaches the enterprise over static routes only; the firewall redistributes the OT umbrella into the corporate IGP rather than letting OT speak it.
3. **Endpoint telemetry follows the Purdue level, not the subnet.** An OT host gets an endpoint agent only if its job is to monitor the process, not to run it. The control-room HMI, the PLC and the field simulator are deliberately left off; the historian and audit DB are instrumented. This is expressed as inventory-group membership so "does this host emit endpoint telemetry" is a decision someone recorded. See the inventory & roles chapter.
4. **Two domains in one forest.** The base enterprise is `blackstone.mil` (forest root) and flight operations is its child `fops.blackstone.mil`. The parent/child trust is automatic and transitive, so identity crosses the two domains by default — which makes the places it should not appear meaningful.
5. **The network is not quiet.** Emulated users browse, open documents and move files continuously across the enterprise and flight-ops segments. Detection logic that assumes a clean baseline will not behave as it does in a lab.

Platform- and library-level decisions (control plane on `10.255.240.0/20`, pfSense firewalls, the OSPF+eBGP+static routing model, role-sourcing policy, the Ubuntu 24.04 baseline) are set by the platform and SimSpace's core Ansible library, and are summarised in the network and build chapters rather than re-argued here.

System architecture

```
          is-inet (200.200.200.2) — simulated Internet / mail / elgg
                              |
          bs-edge-rtr (VyOS, AS 65002)
                              |  eBGP  199.252.163.0/30
          bs-edge-fw (pfSense, AS 65001) ── DMZ 172.31.12.0/24 (bs-www, bs-ftp, bs-ns1)
                              |  172.31.1.4/30
          bs-core-rtr (VyOS, OSPF) ── blackstone.mil enterprise:
                              |          Services  172.31.2.0/24
                              |          HQ        172.31.14.0/24
                              |          IT        172.31.15.0/24
                              |          Supply    172.31.16.0/24
                              |  172.31.1.8/30
          bs-ops-rtr (VyOS, OSPF) ── fops.blackstone.mil:
                              |          Flight-Ops-Services   172.31.3.0/24
                              |          Flight-Control-Center 172.31.5.0/24 (ATC, weather)
                              |          Flight-Ops-Users      172.31.6.0/24
                              |  172.31.1.12/30
          bs-ops-fw (pfSense, OSPF + static L3.5 boundary, default-deny)
              |                               |
    172.31.1.20/30                  172.31.1.16/30 (static only)
              |                               |
    bs-sec-rtr (VyOS, OSPF)          bs-modbus-gateway (VyOS, OT chain head)
      SOC         172.31.7.0/24        MTU            172.16.45.0/29  (control-room-hmi, FUXA)
      Phys-Sec    172.31.11.0/24 (—)   Fuel-Farm PLC  172.16.45.8/29  (ff-plc-1, OpenPLC)
                                       Fuel-Farm Sim  172.16.46.0/24  (fuel-sim, historian, audit DB)
          Engineering 172.31.8.0/24 hangs off bs-ops-fw directly (bs-eng*, bs-ro-hist)
```

Every host is additionally dual-homed onto the **management plane** (`10.255.240.0/20`, gateway/proxy `10.255.240.1`). Ansible connects over management; the data plane never routes to it. The Ansible controller is on the management plane at `10.255.240.157` and is **not** in `[linux]`.

Full addressing and routing detail is in the network chapter.

The bespoke subsystems

- **Base enterprise** — `blackstone.mil`, the forest root: Services (DCs, file, WEC, proxy), Headquarters, IT and Supply workstations, and the DMZ. See the Active Directory chapter.
- **Flight operations** — `fops.blackstone.mil`, the child domain: its own DCs, file and app servers, flight-ops user workstations, and the Flight-Control Center that hosts the ATC and weather subsystem.
- **Air-traffic control** — a Linux subsystem on the Flight-Control-Center segment: a radar/ADS-B replay feeding a controller working position and a set of flight display workstations, with a recorded-weather feed alongside. Built and verified by its own playbook.
- **Fuel farm (OT)** — a Purdue-style enclave behind `bs-ops-fw`, reached only through `bs-modbus-gateway`: an OpenPLC controller, a pymodbus field simulator, a FUXA operator HMI, an in-enclave historian (Telegraf + InfluxDB 2.7 + Grafana) and a PostgreSQL/TimescaleDB audit DB. Modbus TCP is the process protocol.

- **Security / SOC** — the Security Onion grid, analyst workstations and forensic tooling on the SOC segment. On the Splunk-stacked variant a distributed Splunk cluster and a Malcolm OT appliance also live here.

Telemetry / data flows

SOURCE	PATH INTO MONITORING	DATASET
Network (enterprise / flight-ops / OT / edge)	layer-2 `gretap` mirrors → the four sensors → Zeek + Suricata	Zeek `conn` / `dns` / `http` / `files`, Suricata alerts
Windows + Linux endpoints	Elastic Agent → SO Fleet	endpoint (Defend/osquery/system) + Sysmon on the Windows fleet
nginx / Squid / app hosts	agent tails the host's logs	parsed host datasets
pfSense x2 / VyOS x5	devices → `soc-syslog` (rsyslog) → local agent	`logs-pfsense.*`, `logs-vyos-*`
Fuel-farm OT process traffic	second `gretap` tunnel → Malcolm (stacked variant)	Malcolm (Arkime/Zeek ICS parsers)

Everything lands in one monitoring platform, parsed into its own dataset, so correlating a flight-ops login with the process that produced it and the firewall rule that allowed it is one investigation, not three.

Splunk-stacked variant. Every managed host additionally runs a Splunk universal forwarder. Windows event logs and Sysmon, Linux syslog, and the device syslog aggregated on `soc-syslog` are forwarded to the distributed Splunk cluster in parallel with the Security Onion path above. The two monitoring stacks are independent — the same sources feed both — so analysts can be trained and assessed on either.

Artifact sourcing (air-gapped)

The range has no general internet egress. Install artifacts come from the in-platform **Nexus** (`nexus.dev.ng.simspace.lan`), reached through the management proxy: the Security Onion source + container registry, Malcolm's source + images, the Ansible Galaxy collections, the Splunk packages (on the stacked variant), the fuel-farm Python packages, and the HMI's npm closure. **apt is the exception** — base packages and `docker-ce` install from the Ubuntu archive and `download.docker.com` through the proxy, because the Nexus apt repo serves several Ubuntu releases from one index and no pin can safely prefer the right one. Detail in the build & deploy chapter.

02 — Network

All IPs, prefixes and interface assignments are owned by the blueprint and by `host_vars`; this doc is the reconciled map. Re-derive from the platform blueprint after any topology change.

Planes

PLANE	RANGE	NOTES
Production (scenario)	`172.31.0.0/16`, `172.16.45.0/24` + `172.16.46.0/24` (OT), `199.252.163.0/30`, `200.200.200.0/24`	What trainees and sensors see
Management (out-of-band)	`10.255.240.0/20`, gw/proxy `10.255.240.1`	Ansible path; hidden from scenario; mirror-excluded

Every host is dual-homed. The management NIC is in **first** position (Windows `Ethernet0`, Linux `eth0`, pfSense `vmx0`); the production NIC(s) follow. The Ansible controller sits on the management plane at `10.255.240.157`. The management plane shares a data-plane NIC on the mirror-source routers, so it is explicitly excluded from the mirror fabric (see below) and scrubbed from AD DNS (see the Active Directory chapter).

Host-bearing segments

SEGMENT	CIDR	DOMAIN	CONTENTS
DMZ	`172.31.12.0/24`	—	`bs-ftp` (.2), `bs-www` (.3), `bs-ns1` (.5)
Services	`172.31.2.0/24`	blackstone.mil	`bs-dc01` (.7), `bs-dc02` (.8), `bs-file01` (.10), `bs-file02` (.11), `bs-proxy` (.15), `bs-wec` (.17)
Headquarters	`172.31.14.0/24`	blackstone.mil	`bs-hq01..06`
IT	`172.31.15.0/24`	blackstone.mil	`bs-it01..05`
Supply	`172.31.16.0/24`	blackstone.mil	`bs-supply01..05`
Flight-Ops-Services	`172.31.3.0/24`	fops.blackstone.mil	`fops-dc01` (.11), `fops-dc02` (.12), `fops-file01` (.15), `fops-file02` (.16), `fops-wec01` (.20), `fops-app01..03` (.30–.32)
Flight-Control-Center	`172.31.5.0/24`	fops.blackstone.mil	`atc-radar` (.10), `atc-weather` (.11), `atc-station` (.12), `fops-flight01..04`
Flight-Ops-Users	`172.31.6.0/24`	fops.blackstone.mil	`fops-ops01..08`
Engineering	`172.31.8.0/24`	blackstone.mil (Win)	`bs-eng01..06`, `bs-ro-hist` (.5)
SOC	`172.31.7.0/24`	—	SO grid (.15/.20/.21–.24), `soc-syslog` (.13), `soc-openvas` (.16), `soc-sift`, `soc-analyst01..06`; on the stacked variant also the Splunk cluster (.25–.28) and Malcolm (.30)
Physical-Security	`172.31.11.0/24`	—	reserved; no hosts in the blueprint yet

SEGMENT	CIDR	DOMAIN	CONTENTS
OT · MTU	`172.16.45.0/29`	—	`control-room-hmi` (.3, FUXA)
OT · Fuel-Farm PLC	`172.16.45.8/29`	—	`ff-plc-1` (.10, OpenPLC)
OT · Fuel-Farm Sim	`172.16.46.0/24`	—	`fuel-farm-sim` (.17), `fuel-hist` (.18), `fuel-db` (.19)
Internet (sim)	`200.200.200.0/24`	—	`is-inet` (.2), `elgg`

Point-to-point transit links (/30)

LINK	CIDR	ENDPOINTS
edge-rtr ↔ edge-fw	`199.252.163.0/30`	`bs-edge-rtr` .2 ↔ `bs-edge-fw` .1
edge-fw ↔ core-rtr	`172.31.1.4/30`	`bs-edge-fw` .5 ↔ `bs-core-rtr` .6
core-rtr ↔ ops-rtr	`172.31.1.8/30`	`bs-core-rtr` .9 ↔ `bs-ops-rtr` .10
ops-rtr ↔ ops-fw	`172.31.1.12/30`	`bs-ops-rtr` .13 ↔ `bs-ops-fw` .14
ops-fw ↔ modbus-gw	`172.31.1.16/30`	`bs-ops-fw` .17 ↔ `bs-modbus-gateway` .18
ops-fw ↔ sec-rtr	`172.31.1.20/30`	`bs-ops-fw` .21 ↔ `bs-sec-rtr` .22

Routing model

Verified against `host_vars`.

DOMAIN	PROTOCOL	NOTES
ISP edge	**eBGP**	`bs-edge-rtr` (AS 65002) ↔ `bs-edge-fw` (AS 65001). The **only** BGP session in the fabric.
Enterprise + flight-ops core	**OSPF area 0**	`bs-core-rtr`, `bs-ops-rtr`, `bs-sec-rtr` and both pfSense firewalls run OSPF across all transit links and LAN interfaces.
L3.5 OT boundary	**Static, default-deny**	`bs-ops-fw` ↔ `bs-modbus-gateway` is static-only; `bs-modbus-gateway` runs no OSPF. The OT umbrella `172.16.0.0/16` is a static route on `bs-ops-fw` via `172.31.1.18`, and `bs-ops-fw` `redistribute_static` pushes it into corporate OSPF so the enterprise learns the OT prefixes without OT speaking the IGP.

`bs-edge-fw` carries the eBGP session to the ISP; the two pfSense firewalls run **FRR**. `bs-modbus-gateway` is a VyOS router at the head of the OT chain; it is not the same set as the mirror-source routers, though it is one of them.

> The L3.5 boundary is **default-deny by design but permit-any today** — `bs-ops-fw` carries lab "allow any" rules per interface, with blocking ACLs deferred until every subsystem is working. When the ACLs land, the two OT hosts that enrol an Elastic Agent (see the telemetry chapter) need an explicit rule permitting them to reach the SOC subnet on Fleet's ports; the agent preflight fails loudly if that rule is missing rather than opening the path itself.

Firewalls and routers

HOST	TYPE	ROLE IN THE FABRIC
`bs-edge-fw`	pfSense	perimeter; eBGP peer; fronts the DMZ
`bs-ops-fw`	pfSense	OSPF inner core **and** the static-only, default-deny L3.5 OT boundary; gateways the Engineering `/24`
`bs-edge-rtr`	VyOS	the ISP edge (AS 65002); faces the simulated Internet
`bs-core-rtr`	VyOS	gateway for the four enterprise `/24`s
`bs-ops-rtr`	VyOS	gateway for the three flight-ops `/24`s
`bs-sec-rtr`	VyOS	gateway for SOC and Physical-Security
`bs-modbus-gateway`	VyOS	head of the OT chain; gateways the three fuel-farm segments

pfSense specifics (admin-user requirement, `php -r` escape hatches, FRR runtime dir, NIC ordering) are in the operations & troubleshooting chapter and the `pfsense_firewall` role.

DNS

- AD-integrated zones for `blackstone.mil` and the child `fops.blackstone.mil` on their respective domain controllers.
- The `dns` role creates zones/records but does **not** set forwarders; a baseline play points the DCs' forwarders at `is-inet`'s simulated public resolvers so scenario names resolve while the range stays self-contained.
- `bs-ns1` serves the DMZ; `global_dns` provisions scenario name resolution on `is-inet` (the internet simulator's unbound resolver) for the simulated Internet.
- Management-plane A records are scrubbed from AD DNS so `ping <host>` does not resolve to a `10.255.240.x` address.

The GRE mirror fabric

Traffic is copied from router interfaces into `gretap` tunnels with `tc mirred` and delivered to the sensors (and, on the stacked variant, to Malcolm). This is the backbone of positional sensor coverage.

PROPERTY	VALUE
Encapsulation	`gretap` (layer 2). A layer-3 `gre` tunnel yields `DLT_RAW`, which makes Zeek produce **zero** connection logs while Suricata keeps alerting — the classic "sensor looks up but sees nothing" failure. Both ends must be `gretap`.
Tunnel MTU	1500 (raised above the kernel's 1462 `gretap` default); routers clamp TCP MSS so mirrored frames stay within the routed underlay.
Mechanism	`tc` ingress + root `prio` qdiscs per source interface, `matchall action mirred egress mirror dev tun0`. On `bs-modbus-gateway` the OT process interfaces also fan out to `tun1`, feeding Malcolm alongside the sensor.
Persistence	re-applied from the VyOS post-config boot script.
Management exclusion	`10.255.240.0/20` carried by `action pass` filters at lower priority than the `mirred` catch-all, on each source interface.

SENSOR	MIRROR SOURCE	COVERAGE
`soc-sensor-corp`	`bs-core-rtr`	Services, Headquarters, IT, Supply
`soc-sensor-fops`	`bs-ops-rtr`	Flight-Ops-Services, Flight-Control-Center, Flight-Ops-Users
`soc-sensor-ot`	`bs-modbus-gateway`	MTU `/29`, Fuel-Farm PLC `/29`, Fuel-Farm Sim `/24`
`soc-sensor-edge`	`bs-edge-rtr`	the simulated Internet and the edge transit link

Verify a mirror with the absolute path (`tc` is not on the `vyos` user's PATH): `/sbin/tc filter show dev eth1 ingress` — expect one `mirred` filter and the management-exclusion `action pass` filters per source interface. The GRE underlay endpoint each sensor decapsulates from is a per-router fact in `host_vars` (the sensors sit behind `bs-sec-rtr`/`bs-ops-fw`, so the path differs per router and cannot be derived) — do not default it to the sensor's own gateway.

03 — Active Directory

Forest

A single forest with **two domains** and **four** domain controllers:

DOMAIN	ROLE	DC	ADDRESS	INVENTORY GROUP
`blackstone.mil`	Forest root / base enterprise	`bs-dc01`	`172.31.2.7`	`pdc`
`blackstone.mil`		`bs-dc02`	`172.31.2.8`	`additional_dc`
`fops.blackstone.mil`	Child domain / flight operations	`fops-dc01`	`172.31.3.11`	`pdc`
`fops.blackstone.mil`		`fops-dc02`	`172.31.3.12`	`additional_dc`

`fops.blackstone.mil` is a **child domain** in the same forest as the root `blackstone.mil`. The parent/child trust is automatic, bidirectional and transitive — no manual cross-forest trust is set up, and a principal in either domain can authenticate to resources in the other by default. Routine cross-domain identity traffic is therefore expected; the point of interest is where it appears that it should not.

Promotion order matters. The two forest-root and child-root DCs (`bs-dc01`, `fops-dc01`) are promoted first via `dcpromo`; the additional DCs (`bs-dc02`, `fops-dc02`) join via `additional_dc`. The child domain is the fragile step: a half-completed child promotion leaves residue that blocks a clean retry, so a `dcpromo(child)` self-heal pass runs ahead of promotion to clear it.

Windows Event Collectors

Each domain has a WEC that receives forwarded Windows events and is the audited log path into monitoring:

WEC	ADDRESS	DOMAIN
`bs-wec`	`172.31.2.17`	blackstone.mil
`fops-wec01`	`172.31.3.20`	fops.blackstone.mil

Membership

- **44 domain members** (`[members_blackstone]` 29 + `[members_fops]` 15) + 4 DCs = **48 domain-joined hosts**. Every Windows VM in the range is domain-joined.
- Enterprise members include the HQ/IT/Supply workstations, the file servers, the Engineering **Windows** workstations (`bs-eng01..04`) and the SOC analyst workstations (`soc-analyst01..06`) — Engineering and SOC Windows hosts join `blackstone.mil` by decision.
- Flight-ops members include the flight-ops user workstations, the flight display workstations, the file servers and `fops-wec01`.
- The Engineering **Linux** hosts (`bs-eng05`, `bs-eng06`), the ATC subsystem hosts, the fuel-farm OT hosts, the Security Onion grid and the forensic tooling are **not** domain-joined.

Counts are derived from the inventory; re-derive after inventory changes (see the inventory & roles chapter).

Emulated users

Domain user identities with realistic names are seeded across both domains and placed in the appropriate groups. Emulated users run on the Windows workstations in `[aue]` — HQ, IT, Supply, Engineering (Windows), flight-ops users, the flight display consoles, and the SOC analyst workstations — and log in, browse, open documents and move files continuously. This is the background noise detection logic must survive. `is-inet` hosts mailboxes for both domains (Postfix + Dovecot, multi-domain) so user mail traffic is real.

DNS and the management plane (the recurring hazard)

Every host is dual-homed, with a management NIC on `10.255.240.0/20`. Windows registers **both** adapters into AD DNS by default, so `ping <host>` can resolve to a management IP half the time. Two mitigations are wired in:

1. A baseline play strips the management interface from AD DNS registration and scrubs `10.255.240.x` A records. (The base `common` role's own attempt at this has known bugs — a typo'd adapter name and the wrong cmdlet parameter — so the overlay play is the real fix.)
2. DCs force-register and prove their locator SRV records before any member join runs, because a fresh build sometimes starts joins before Netlogon has registered them.

`Resolve-DnsName <host> -DnsOnly` is the gold-standard check — `ping` and `nslookup` both have LLMNR/NetBIOS footguns on the shared management L2.

DNS forwarders

The `dns` role does not set forwarders, so anything outside the two Blackstone domains is unresolvable on domain workstations until a baseline play points the DCs at `is-inet`'s simulated public resolvers. This keeps the range self-contained while still giving trainees name resolution for scenario domains.

Child-DC / DNS-partition convergence note

On a cold build a freshly promoted child DC can answer ADWS minutes before its `DomainDnsZones` application partition is enlisted, and DNS record writes against it fail until it is. The `dns` play gates on the partition being present (probing it, creating the built-in partitions if absent, restarting the DNS service, re-probing) and record writes retry rather than failing the host. Keep the gate and the record retry if you refactor the `dns` play — the two-domain build exercises this path on every cold run.

04 — Inventory and Roles

How membership drives behaviour

The `hosts` INI inventory is the control surface for the whole build. Group names are **load-bearing**: roles and plays address them by name (`groups['so_manager']`, `hosts: windows`, `windows:linux:!so_all:!no_endpoint_agent`), so renaming a group in `hosts` without renaming it in the roles breaks the build silently. A host's capabilities and the telemetry it emits are decided entirely by which groups it is in.

Two naming choices are deliberate and worth internalising:

- There is **no `[unmanaged]` group.** The OT hosts the enterprise would call "unmanaged" are fully Ansible-managed here — the group is named `[no_endpoint_agent]` because the only thing those hosts are excluded from is endpoint telemetry, not management.
- There is **no generic `[siem]` umbrella group.** Used as an exclusion it would silently change meaning every time a host joined it; plays name `[so_all]` directly instead.

Inventory groups (current expanded sizes)

Platform / OS groups:

GROUP	SIZE	MEANING
`windows`	48	all Windows VMs (`:children` of `win10` 15 / `win11` 23 / `winserver2022` 10)
`linux`	26 (30 stacked)	`:children` of `ubuntu22` / `ubuntu24` / `ubuntu25`; the stacked variant adds the four Splunk VMs
`vyos`	5	`bs-edge-rtr`, `bs-core-rtr`, `bs-ops-rtr`, `bs-sec-rtr`, `bs-modbus-gateway`
`pfsense`	2	`bs-edge-fw`, `bs-ops-fw`

Active-Directory groups:

GROUP	SIZE	MEANING
`pdc` / `additional_dc` / `domain_controllers`	2 / 2 / 4	DC promotion roles across both domains
`members_blackstone` / `members_fops`	29 / 15	domain-joined non-DC hosts per domain
`wec`	2	Windows Event Collectors (`bs-wec`, `fops-wec01`)

Role / posture groups:

GROUP	SIZE	MEANING
`aue`	38	emulated-user workstations
`ae`	10	attack-emulation targets: both domains' DCs, file servers and WECs
`fileservers`	4	`bs-file01/02`, `fops-file01/02`
`no_endpoint_agent`	3	OT hosts that run the process, so get **no** endpoint agent: `ff-plc-1`, `fuel-farm-sim`, `control-room-hmi`
`ot_fuel` / `ot_mtu`	4 / 1	fuel-farm OT hosts; the MTU HMI segment

GROUP	SIZE	MEANING
`atc`	3	ATC subsystem Linux hosts (`atc-radar`, `atc-weather`, `atc-station`)
`soc_analysts`	6	analyst workstations (`soc-analyst01..06`)
`dmz`	3	`bs-www`, `bs-ftp`, `bs-ns1`

Security Onion groups:

GROUP	SIZE	MEANING
`so_manager` / `so_search` / `so_sensor`	1 / 1 / 4	grid node roles
`so_all` (`:children`)	6	every grid node
`so_mirror_routers`	4	the VyOS routers carrying a GRE mirror (`bs-core-rtr`, `bs-ops-rtr`, `bs-modbus-gateway`, `bs-edge-rtr`)
`ansible_controller`	1	the mirror/controller host — `ansible_connection=local`, deliberately **not** in `[linux]`

Splunk + Malcolm groups (stacked variant only):

GROUP	SIZE	MEANING
`splunk_search_head` / `splunk_indexer` / `splunk_cluster_manager`	1 / 2 / 1	the distributed Splunk cluster
`splunk-forwarder` (`:children` = windows + linux)	78	universal-forwarder recipients
`splunk_forwarder_exempt`	—	the Splunk tier itself + `so_manager` + `so_search`
`malcolm`	1	the Malcolm OT appliance (`soc-malcolm`)

The key derived numbers today: **85 managed hosts** on the monitoring-only variant (**89** on the stacked variant); **48 Windows**; **48 domain-joined** (44 members + 4 DCs); **66 Elastic-Agent endpoints** (`[windows]`+`[linux]` minus `[so_all]` minus `[no_endpoint_agent]`). Sizes are computed from the current inventory and move when hosts are added/removed; the local `ansible` control node is counted separately from the managed total.

Why the control node must be `ansible_connection=local`

`[ansible_controller]` holds exactly one host, and it carries `ansible_connection=local`. Without it the controller inherits the default SSH connection and Ansible tries to SSH to itself; that one host returns `unreachable`, the group has no other member, and the run ends with "NO MORE HOSTS LEFT" at the first phase — every later phase silently never executes, with `failed=0` on every host because nothing that could fail was reached. The controller's `ansible_host` is still its management IP: connection-local and reachable-at-that-IP are different facts about the same host, and the SO nodes fetch the in-range mirror from that IP.

`group_vars` hierarchy (lowest → highest precedence)

1. `group_vars/all/` — `main.yml` (proxy, syslog server IP, domain-admin account, shared tunables), `security_onion.yml` (the entire SO stack: pins, registry, subnets, airgap + Nexus source URLs), `vault.yml` (encrypted secrets).

2. `group_vars/<group>.yml` — `windows.yml`, `linux.yml`, `vyos.yml`, `pfsense.yml`, `blackstone.yml`, `fops.yml`, `fuel.yml`, `atc.yml`, and (on the stacked variant) `ubuntu25.yml`, `malcolm.yml`, `splunk_indexer.yml`, `splunk_search_head.yml`.
3. `host_vars/<host>.yml` — per-host IPs, interfaces, role-specific tuning.
4. Inline play vars in `site.yml`.

No hardcoded IPs live in tasks or templates — `group_vars` / `host_vars` are the single source of truth.

Roles

Roles are pulled from SimSpace's core Ansible library first, then overridden by the local overlay roles (an overlay role of the same name **replaces** the base role wholesale). The tarball bundles the base roles it uses plus the custom overlays; the exact set is auto-discovered from every play (see the build & deploy chapter). The monitoring-only variant carries 52 roles; the stacked variant adds 11 (Splunk x8, Malcolm x2, `docker_engine`).

Range / enterprise roles

ROLE	PURPOSE
`init`	Waits for WinRM, then repairs and pins the default-gateway ARP entry (an impostor MAC answers ARP/ICMP for the gateway on some segments).
`common`	Overlay of the base baseline role.
`dcpromo` / `dcpromo_child_heal`	Promote the forest-root and child-domain DCs; the heal pass clears half-join residue before a child promotion retries.
`additional_dc`	Joins the second DC in each domain.
`domain_member_retry`	Robust domain join with repair passes — the join path, not the member, is usually what fails on a cold build.
`dns`	AD-integrated zones + records; child-DC `DomainDnsZones` partition gate; record writes retry.
`global_dns`	Scenario public-name resolution on `is-inet` for the simulated Internet.
`create_users` / `ue` / `aue_agent`	Seed domain users; drive emulated-user activity.
`fileserver` / `mapped_drive`	File servers and the mapped drives that reference them.
`email`	Postfix + Dovecot + webmail on `is-inet`, multi-domain for both forests.
`blackstone_www`	The public web presence on `bs-www`.
`squid` / `platform_proxy`	The in-scenario proxy (`bs-proxy`) and platform-proxy wiring.
`strip_apipa`	Removes `169.254.x.x` APIPA addresses Windows assigns during DHCP→static handoff.
`network_discovery` / `disable_firewall` / `disable_uac` / `autologin` / `chrome` / `secpol` / `root_certs` / `ae_gpo`	Windows posture and workstation enablement.
`pfsense_firewall`	

ROLE	PURPOSE
	Drives pfSense 2.8.1 via `pfsensible.core` + `php -r` escape hatches (interfaces, gateways, default-gw pin, OSPF/BGP, static routes, rules, syslog).
`vyos` / `vyos_mirror`	VyOS router config; GRE `gretap` tunnels + `tc mirred` mirror rules feeding the sensors (and Malcolm).
`syslog_server`	`soc-syslog` as the central rsyslog collector (per-device files + agent fan-out).

OT / fuel-farm roles

ROLE	PURPOSE
`fuel_plc`	OpenPLC v4 controller on `ff-plc-1` (Modbus TCP `:502`).
`fuel_sim`	pymodbus field simulation on `fuel-farm-sim` — it is the plant.
`fuel_hmi`	FUXA operator console on `control-room-hmi` (`:1881`); the Modbus-serial driver installs into the container from the Nexus npm group.
`fuel_historian`	Telegraf + InfluxDB 2.7 + Grafana on `fuel-hist`.
`fuel_db`	PostgreSQL 16 + TimescaleDB audit DB on `fuel-db`.
`drainhole`	Fuel-process support.

ATC roles

ROLE	PURPOSE
`atc_core`	The radar/ADS-B replay and controller working-position backend.
`atc_cwp`	Controller working position / display on `atc-station` and the flight display clients.
`atc_inject`	The recorded flight-data injector.
`atc_weather`	The recorded-weather feed on `atc-weather`.

Monitoring roles

ROLE	PURPOSE
`so_apt_mirror`	nginx on the controller serving the SO source snapshot, airgap content, ETOPEN rules and the skopeo-built container registry.
`so_base`	Prereqs on every grid node before `so-setup`.
`so_manager`	Renders the answer file, seeds the registry from the mirror, runs `so-setup`, verifies the outcome (not the exit code).
`so_search`	Search-node install + grid join.
`so_sensor`	Sensor install + grid join, GRE decap, Zeek/Suricata.

ROLE	PURPOSE
`so_fleet_integrations`	Host-scoped Fleet policies + integrations (pfSense, VyOS, host logs) driven directly against the Fleet API.
`elastic_agent`	Installs/enrols the Elastic Agent on every endpoint from controller-staged installers.
`sysmon`	Installs Sysmon across the `[windows]` fleet from a Nexus-staged installer.
`docker_engine` (stacked)	Installs Docker honouring the Nexus insecure-registry/proxy config (consumed by Malcolm).
`malcolm` / `malcolm_host_prep` (stacked)	Deploys the Malcolm OT appliance (26.08.0, images from Nexus retagged to `ghcr.io`) and prepares its `gretap` decap interface.
`splunk` / `splunk_cluster_manager` / `splunk_indexer` / `splunk_search_head` / `splunk_license_peer` / `splunk-forwarder` / `splunk-es` / `splunk-verify-user` (stacked)	The distributed Splunk cluster, the universal forwarders, Enterprise Security content, and a post-deploy login/search proof.

Adding a role

Don't edit the role list in `build_tarball.sh` — it auto-discovers from plays. Add a role by referencing it in a play (`roles:`, `import_role` or `include_role`), and it is bundled on the next build.

05 — Build and Deploy

The deploy chain, end to end

```
build_tarball.sh  ──>  ab_mb.tgz (or ab_mbs.tgz, stacked)  ──>  commit + push (pinned commit)
                                          |
blueprint download-range-vars-script: wget the tarball from the pinned raw URL
                                          |  (through mgmt proxy 10.255.240.1:3128)
                                  tar -xzf into /etc/ansible
                                          |
blueprint deploy-script:  ./deploy.sh | tee /var/log/playbook_run.log
                                          |
                                   site.yml, up to 3 attempts
```

The controller **deploys from the tarball, not a git pull**. The repo is just where the tarball is built and versioned; the running range only ever sees the tarball (`ab_mb.tgz` on the monitoring-only variant, `ab_mbs.tgz` on the stacked variant). Fetch files on the controller with `wget` through the mgmt proxy — `curl` does not work through it.

Building the tarball

```
./build_tarball.sh          # writes the tarball; refuses if a gate fails
```

`build_tarball.sh` auto-discovers roles from **every** play under `playbooks/` plus `site.yml` and the top-level `fuel_farm_playbook.yml` / `atc_playbook.yml`, understanding `roles:`, `import_role` and `include_role` (so a role referenced only via `import_role` + `tasks_from`, like `vyos_mirror`, is still bundled). Each role is taken from SimSpace's core Ansible library first, then overridden by the local overlay roles. Bundled artifacts: roles, playbooks, `site.yml`, `hosts`, `group_vars`, `host_vars`, the verifier scripts, `deploy.sh`, `requirements.yml`, the vendored `collections/`, the ETOPEN `rules/`, **and** the encrypted vault plus its password file (see "Secrets / vault" below). **Not** bundled: `docs/`.

Build-time gates (the archive is refused if any fails)

GATE	CATCHES
`verify_shell_args.py`	An apostrophe in a PowerShell/shell comment inside a free-form module arg — Ansible's `split_args()` counts quotes and the play fails to **load** (valid YAML, so a separate check).
`verify_so_inventory.py`	An SO host in `[so_all]` but not `[linux]` (roles to run, no way to log in — it once cost three deploy attempts).
`verify_dup_keys.py`	Duplicate YAML mapping keys (silently drop data).
`verify_handlers.py`	`notify:` targets that resolve to no handler.
`verify_task_keywords.py`	Task keywords misindented as module arguments.
`verify_defend_filters.py`	Malformed Elastic Defend filters.
`verify_vars.py`	

GATE	CATCHES
	Jinja references with no definition. Advisory, not a hard gate. Run it against the **stage**, not the repo root — the stage includes base roles copied from SimSpace's core Ansible library, so some vars appear only there.

Publishing

Rebuild the tarball **and** commit it with the code change, every time, because the controller deploys the tarball. After committing, **verify the committed artifact, not the working copy**:

```
git show HEAD:ab_mb.tgz | tar tz | head                      # it is really in the commit
git show HEAD:ab_mb.tgz | tar xzO ./hosts | grep -c <thing>   # spot-check content
```

Hand over a **commit-pinned** raw URL (not a branch URL — branch URLs are CDN-cached), then point the blueprint's `download-range-vars-script` at that SHA (see the operations & troubleshooting chapter).

`deploy.sh` — the 3-attempt retry model

`deploy.sh` runs `site.yml` up to three times, and **attempt 2 is not a success condition**:

1. full sweep
2. retry-file scope — a **repair pass** over the hosts that failed in attempt 1
3. full sweep — **this is what actually confirms the range**

The retry file lists hosts that failed. Repairing them does not re-run the plays whose targets were dropped when they failed, so a clean repair pass is not a deployed range. Attempt 3 is the confirmation. A first-attempt failure is often ordering (a host depends on state another reaches asynchronously), not a fault.

- `FORKS` is **derived**, not chosen: the largest single play target plus a small margin, so the widest play runs in one batch. Recount when hosts are added.
- `BOOT_DELAY` (default 180s; `BOOT_DELAY=0` to skip) lets a freshly provisioned range finish booting — it covers hosts that do not exist yet, which `wait_for_connection` cannot. Use `BOOT_DELAY=0 ./deploy.sh` on an already-up range.

```
sudo ./deploy.sh                # full build, 3 attempts
BOOT_DELAY=0 ./deploy.sh        # already-up range
ansible-playbook playbooks/40-manager.yml           # one phase, during dev
ansible-playbook site.yml --limit bs-dc01,soc-so-manager  # targeted re-run
```

Phase order (`site.yml`)

`site.yml` holds the range **baseline** as inline plays (init + common across all hosts, both AD domains + the child trust, DNS, DHCP, the firewalls and routers, the enterprise and flight-ops tiers) and then imports the monitoring and subsystem phases in order:

```
10-mirror          GRE/tc mirror + SO source staging — imported FIRST (fails
                   fast; kicks the 7 GB registry build off in the background)
  ...                the inline baseline plays (AD, DNS, firewalls, routers,
                   enterprise, flight-ops; on the stacked variant also the
                   whole Splunk cluster)
```

```
05-time              Windows clock, DC-first
20-vyos              GRE tunnels + tc mirror rules on the four mirror routers
30-prereqs           so_base on all SO nodes
40-manager           SO manager install — MUST complete before 50
50-nodes             search + sensor grid-join
60-verify            grid verification
70-analyst           analyst-workstation enablement
75-endpoint          Sysmon, then Elastic Agent enrolment
80-fleet-integrations  move log-source hosts onto dedicated Fleet policies
90-malcolm           Malcolm OT appliance (stacked variant only)
fuel_farm_playbook   the fuel-farm OT chain
atc_playbook         the ATC subsystem
```

`10-mirror` is hoisted ahead of the baseline because the mirror/registry is a hard prerequisite for 30/40/50 — discovering it broken after a multi-hour baseline wastes the run — and because it overlaps the 7 GB image build with the baseline. Safe to hoist because the controller is in `[ansible_controller]` only, never `[linux]`.

`75-endpoint` runs after `60-verify` because enrolment needs a manager that is actually green — Fleet has to be answering before an agent can register — and `80-fleet-integrations` **must** follow it: enrolment puts every endpoint on `endpoints-initial`, then that phase reassigns the log-source hosts to dedicated policies. On the stacked variant the Splunk universal forwarder is laid down in the inline baseline and the Elastic Agent in `75-endpoint`; the two read the same sources independently and neither can starve the other.

Secrets / vault

Both variants carry an **ansible-vault**–encrypted `group_vars/all/vault.yml`. `deploy.sh` enforces it with a **fail-closed guard**: it refuses to run unless `group_vars/all/vault.yml` exists and is `$ANSIBLE_VAULT`-encrypted, and unless the password file is present. On this range the vault password ships **inside the tarball** alongside the encrypted vault (the whole tarball is the trust boundary), so a fresh controller needs no out-of-band step. The vault holds the Security Onion web/join secrets and the OT (OpenPLC/FUXA/InfluxDB/fuel-DB) and — on the stacked variant — Malcolm secrets. The OS/device connection accounts and the domain-admin account live in the per-group `group_vars` as described in the operations chapter. Tasks that embed a credential in a shell command carry `no_log`, because Ansible echoes `cmd` on failure and `deploy.sh` tees that to `/var/log/playbook_run.log`.

Artifact sourcing (Nexus)

The range has no general egress. Everything is fetched from the in-platform Nexus (`nexus.dev.ng.simspace.lan`) through the management proxy (`10.255.240.1:3128`); Nexus presents an internal-CA cert, so each fetcher is told to trust or skip-verify that host:

ARTIFACT	NEXUS REPO	FETCHER / NOTE
SO source snapshot + airgap content	`ng_raw` (`installers/so/...`)	`get_url`, `validate_certs: false`
SO + Malcolm container images	`nexus-docker`	`skopeo --src-tls-verify=false`; dockerd `insecure-registries`
Malcolm 26.08.0 source	`ng_raw` (`installers/malcolm/...`)	`get_url` (stacked)

ARTIFACT	NEXUS REPO	FETCHER / NOTE
Ansible Galaxy collections	`ng_raw/galaxy`	`ansible-galaxy ... --ignore-certs`
Fuel-farm Python packages	`ng_pypi`	`pip --index-url ... --trusted-host ...`
HMI npm closure (`modbus-serial`)	`npm` group	`npm --registry ... --strict-ssl=false`, via the mgmt proxy
Splunk packages	`ng_raw`	staged `.deb` (stacked)
apt packages + `docker-ce`	**Ubuntu archive / `download.docker.com`**	**NOT Nexus.** `ng-apt` serves several Ubuntu releases from one index; no pin can safely prefer the right one, so apt stays on upstream through the proxy.

Before cutting a blueprint over to a Nexus-sourced tarball, confirm Nexus is populated and reachable through the proxy for that range — otherwise the deploy fails early on the SO source or a Galaxy fetch.

06 — Security Monitoring and Telemetry

Security Onion is the range's network-security monitoring grid; every log source reports to it. On the **Splunk-stacked variant** a distributed Splunk cluster runs alongside it, fed in parallel by a universal forwarder on every managed host, and a Malcolm appliance adds ICS-protocol deep-packet analysis for the fuel-farm enclave (see the later sections); the monitoring stacks are independent.

The grid

Distributed Security Onion 2.4 on the SOC segment (`172.31.7.0/24`):

NODE	ADDRESS	ROLE
`soc-so-manager`	`172.31.7.15`	Manager, SOC web UI (`https://172.31.7.15`), Fleet
`soc-so-search`	`172.31.7.20`	Search node (`SEARCHNODE`)
`soc-sensor-corp`	`172.31.7.21`	Enterprise traffic
`soc-sensor-fops`	`172.31.7.22`	Flight-ops traffic
`soc-sensor-ot`	`172.31.7.23`	Fuel-farm OT traffic
`soc-sensor-edge`	`172.31.7.24`	Attack path + simulated Internet

Pins live in `group_vars/all/security_onion.yml`:

VAR	VALUE	MEANING
`so_git_ref`	`55af7eb...`	SO 2.4/main HEAD snapshotted 2026-07-22 (a SHA, not a tag — the answer-file mechanism ships only on 2.4/main)
`so_registry_image_version`	`2.4.211`	SO image set
`so_registry_registry_version`	`3.0.0`	registry container
`so_registry_elasticsearch_version`	`9.0.8`	Elasticsearch
`so_airgap_mode`	`true`	no internet during setup
`so_registry_min_repos`	`22`	setup is gated on this many repos present in the local registry

The SOC web UI is configured for **IP access** (`BASIC` advertisement), so there is no hostname to resolve — reach it from an analyst workstation at `https://172.31.7.15`, admin `admin@blackstone.mil`. The grid runs on Ubuntu Desktop 22.04.5: SO 2.4 at the pinned SHA refuses `noble` outright and only tolerates `jammy` because `so_base` fakes it as `focal`, which is why these nodes sit in `[ubuntu22]` while the base Linux baseline is 24.04.

Airgap install: the registry seed

`so-setup` runs in airgap mode and pulls container images from a **local registry**, never ghcr/dockerhub. The flow:

1. `so_apt_mirror` (on the controller) serves the SO source snapshot, the airgap content, the ETOPEN ruleset, and the container-registry artifacts.

2. The registry images are staged as `registry.tar` / `registry_image.tar` **before** `so-setup` runs — `so-setup` deletes the docker proxy drop-in mid-run, so anything not already local is unreachable.
3. `so_manager` seeds the registry, renders the answer file, runs `so-setup`, and verifies the **outcome** (repos present, services up) rather than the exit code. Setup is gated on `so_registry_min_repos` (22) being present.

With `so_registry_source` pointing at the Nexus docker registry the controller pulls the registry contents from Nexus; with `so_registry_artifact_url` set to a raw base it downloads a prebuilt tarball; otherwise it builds the registry locally with skopeo. No in-play system needs internet access.

Positional sensor coverage

Each sensor is fed by exactly one router's `gretap` mirror (see the network chapter for the `tc`/`gretap` mechanics). There is no aggregation tap, so:

- An event on `soc-sensor-ot` and absent from `soc-sensor-fops` is traffic that did **not** cross the OT boundary.
- Cross-segment activity must be correlated across sensors, not read from one stream.
- The management plane is excluded from every mirror (`action pass` for `10.255.240.0/20` ahead of the `mirred` catch-all), so trainees never see the orchestration plane.

SENSOR	MIRROR SOURCE	SEGMENTS
`soc-sensor-corp`	`bs-core-rtr`	Services, HQ, IT, Supply
`soc-sensor-fops`	`bs-ops-rtr`	Flight-Ops-Services, Flight-Control-Center, Flight-Ops-Users
`soc-sensor-ot`	`bs-modbus-gateway`	MTU, Fuel-Farm PLC, Fuel-Farm Sim
`soc-sensor-edge`	`bs-edge-rtr`	simulated Internet + edge transit

The single most common sensor failure is a `gre` (layer-3) tunnel where a `gretap` (layer-2) tunnel is required: Suricata keeps alerting while Zeek produces **zero** `conn.log` records. Both tunnel ends must be `gretap`.

Endpoint telemetry

`elastic_agent` enrols every `[windows]` and `[linux]` host into SO's Fleet from controller-staged installers — **66 endpoints**: every Windows and Linux host except the SO grid (`[so_all]`, which runs its own agent) and the three OT hosts in `[no_endpoint_agent]`. **Sysmon** runs across the entire `[windows]` fleet (48 hosts) from a Nexus-staged installer.

The OT exclusion is the instructive one. Endpoint-agent placement in the OT chain follows the **Purdue level, not the subnet**: a host gets an agent only if its job is to monitor the process, not to run it.

- `ff-plc-1` (L1, the OpenPLC controller), `fuel-farm-sim` (L0/1, the field simulation) and `control-room-hmi` (L2, the FUXA operator console) get **no** agent — they are the control loop, and `control-room-hmi` is additionally a Purdue bridge host where an agent's failure mode is worst.
- `fuel-hist` (L3 historian) and `fuel-db` (L3 audit DB) **are** agent targets — Level 3 records and reports, it does not drive the process, and endpoint agents are routine there.

Excluding the HMI is a deliberate, conservative reading for a training range: it leaves a real endpoint-visibility gap in the OT enclave that trainees have to close with network monitoring instead. `soc-sensor-ot` mirrors all three OT segments, so the traffic is still fully visible on the wire — that is the compensating control, and teaching it is the point.

Because the two agent-bearing OT hosts cross the L3.5 boundary, their permitted Fleet path is expressed as two `/32`s in `so_agent_endpoint_subnets` (not a `/24` that would also admit `fuel-farm-sim`), and `75-endpoint` asserts every agent target's address falls inside the permitted set — add a host on a new segment and the assert fails loudly instead of that host silently failing to enrol.

Application and device log sources

Non-endpoint sources reach SO as **parsed datasets**, each on a dedicated Fleet agent policy created by `so_fleet_integrations`. The firewalls and routers cannot run an agent, so `soc-syslog` is the central rsyslog collector for them:

- **pfSense x2.** rsyslog on `soc-syslog` relays each firewall's raw datagram to a per-firewall loopback port — `bs-edge-fw` → 9001, `bs-ops-fw` → 9002 — where the Elastic pfSense integration parses it. The port carries the device identity, because pfSense omits the hostname field on remote forwards and the loopback relay would otherwise stamp every message with the same source address. Nothing re-renders the message, so grok parsing is untouched.
- **VyOS x5.** The agent tails `soc-syslog`'s `/var/log/remote/<router>/syslog.log` directly. The VyOS paths are **enumerated per router, never globbed** — `/var/log/remote/` also holds the SO grid nodes' own daemon logs (they are in `[linux]`), and SO already collects those natively, so reading them back would be duplicate data about the thing doing the reading.

Fleet-integration rules worth knowing

- Derive the input id, never pattern-match it: the key is `<policy_template>-<input_type>`, and a package's policy template is not always named after the package.
- A dedicated policy contains only what you put in it, so moving a host off `endpoints-initial` strips its endpoint telemetry unless the base integrations are mirrored too (`mirror_base: true`).
- Renaming an integration does not replace the old one (`..._check` matches by name), so add the old name to the retired-integrations list in the same change.
- `so_fleet_integrations` drives `elastic_fleet_integration_create/_update` directly rather than `so-elastic-fleet-integration-policy-load`, which walks only hardcoded directories and would never read one we add.

Splunk cluster (stacked variant)

On the Splunk-stacked variant, a distributed Splunk Enterprise cluster runs on the SOC segment alongside the Security Onion grid, giving analysts a second, independent SIEM over the same estate:

NODE	ADDRESS	ROLE
`soc-splunk`	`172.31.7.25`	Search head (analyst-facing web UI)
`soc-splunk-idx01`	`172.31.7.26`	Indexer (cluster peer)
`soc-splunk-idx02`	`172.31.7.27`	Indexer (cluster peer)
`soc-splunk-cm`	`172.31.7.28`	Cluster manager + license manager

The two indexers form a single indexer cluster (label `mb-idxc`, replication and search factor 2); the cluster manager distributes `indexes.conf` and is the one license manager — `splunk_license_peer` points the manager, both peers and the search head at it, because four instances each as their own license manager is four violations, not one deployment. Collection is by **universal forwarder on every managed host** (`[splunk-`

`forwarder]` = `[windows]` + `[linux]`, 78 hosts), with the Splunk tier itself plus `so_manager`/`so_search` carved out by `[splunk_forwarder_exempt]`:

- Windows hosts forward event logs and Sysmon; Linux hosts forward syslog and service logs.
- The firewalls' and routers' syslog is aggregated on `soc-syslog` and a forwarder there ships it to the indexers.
- The Splunk cluster hosts are themselves Elastic-Agent targets — an indexer is exactly the kind of host worth watching — which is why a path-scoped Defend exclusion for `/opt/splunk/var` exists, so their own bucket churn does not swamp the dataset.

Because the forwarder path and the Elastic Agent path are independent, the same source feeds both stacks — an analyst can be trained or assessed on either, and a gap in one is not a gap in the other. `splunk-verify-user` proves after deploy that a user can actually log in and search, because a search head that cannot search is not a working SIEM.

Malcolm — OT deep-packet analysis (stacked variant)

`soc-malcolm` (`172.31.7.30`, SOC segment) is a CISA Malcolm appliance dedicated to the fuel-farm enclave. Security Onion sees the same OT traffic on `soc-sensor-ot`; Malcolm adds the ICS parsers (Modbus and the rest) that make process traffic legible.

PROPERTY	VALUE
Version	Malcolm **26.08.0** (OpenSearch, Arkime, Zeek, Suricata, Logstash) under Docker
Image source	pulled from the Nexus docker registry and **retagged locally to their `ghcr.io` names** — the compose stays `ghcr.io` because Malcolm's keystore step greps the compose for those names; rewriting the refs breaks keystore creation and the stack never starts
Source	`ng_raw` tarball, unpacked into the install dir (gates the git clone)
Capture	live on `otmir0` — an IP-less `gretap` decap interface; Zeek/Suricata/netsniff-ng read the decapsulated mirror
Feed	a second `gretap` tunnel (`tun1`) fanned out from `bs-modbus-gateway`, carrying the fuel-farm process segments, aimed at `172.31.7.30`
Web UI	`https://172.31.7.30`

The appliance is **not** in `[linux]`, so the base baseline never runs against it; the Malcolm phase owns its whole configuration. The tunnel is receive-only — Malcolm observes the mirrored flows, it never replies.

07 — Verification

Verification happens in two places: **build time** (gates in `build_tarball.sh` that refuse to ship a bad archive) and **runtime** (checks after a deploy that assert outcomes, not exit codes).

Build-time gates

Covered in the build & deploy chapter. In short, `build_tarball.sh` runs `verify_shell_args.py`, `verify_so_inventory.py`, `verify_dup_keys.py`, `verify_handlers.py`, `verify_task_keywords.py`, `verify_defend_filters.py` and `verify_vars.py`, and aborts the build if a hard gate fails. Run `verify_vars.py` against the **stage**, not the repo root — the stage includes base roles copied from SimSpace's core Ansible library, so some vars appear only there and a count taken at the repo root misleads.

Runtime verification

The SO grid is verified **in the deploy** by the `60-verify` phase — `so-status` on the manager, `/_cluster/health` from the manager, and a real packet count on every sensor's `tun0`. There is deliberately no standalone `verify_so.sh`: it would duplicate those checks against a hardcoded host list, and the checks that matter are the ones that run in the deploy.

Three read-only scripts run on the controller after a deploy:

```
sudo ./verify_deployment.sh     # range-wide health check
sudo ./verify_fuel_farm.sh      # fuel-farm OT acceptance
sudo ./verify_atc.sh            # ATC subsystem chain
```

`verify_deployment.sh` is the broad one. It checks per-group reachability over the management plane; asserts the impostor-gateway MAC is absent from every Windows host's ARP table; counts AD domain-join per group across both domains; confirms the DCs' DNS forwarders point at `is-inet`; checks that `bs-ops-fw`'s FRR RIB and the kernel FIB agree on the enterprise prefixes (a zebra-poisoning guard); and queries Security Onion's Elasticsearch to confirm the device and host log sources are actually ingesting. It prints `Total / Pass / Fail` and exits non-zero if anything failed — **the number that matters is `Fail: 0`.**

`verify_fuel_farm.sh` is the fuel-farm acceptance check (historian, audit DB, PLC and HMI reachability and function). `verify_atc.sh` walks the ATC subsystem chain; both use `wget`, never `curl`, because `curl` does not work through the controller's proxy.

Then confirm the thing that most often fails silently — that sensors are seeing traffic, not merely running:

```
sudo ansible soc-sensor-corp,soc-sensor-fops,soc-sensor-ot,soc-sensor-edge -b -m shell \
  -a 'timeout 10 tcpdump -i tun0 -c 20 2>&1 | tail -3'
```

A sensor with Suricata alerting but Zeek producing zero connection logs means the tunnel is carrying the wrong frame type — both ends must be `gretap` (see the security monitoring & telemetry chapter). Verify a mirror itself with the absolute path (the `vyos` user has no `tc` on PATH, and a bare invocation returns "command not found" that a counting grep cannot tell from "no filters"):

```
/sbin/tc filter show dev eth1 ingress
```

What "healthy" looks like

CHECK	EXPECTED
`so-status` on every SO node	all services running
Elastic cluster	green, search node present
`tcpdump -i tun0` on each sensor	packets flowing
Zeek `conn.log`	connection records, not zero
Fleet	every endpoint enrolled (66 agent targets)
Domain membership	44 members joined across the two domains (48 counting the 4 DCs)
Local registry (airgap)	>= 22 repos before `so-setup` proceeds
Fuel farm	`verify_fuel_farm.sh` clean
ATC	`verify_atc.sh` clean

Three-layer post-deploy check

L1 — routing convergence

On each VyOS router:

```
show ip ospf neighbor   # adjacencies Full on the corp routers
show ip bgp summary     # only bs-edge-rtr ⟷ bs-edge-fw is Established
show ip route 0.0.0.0/0 # default in FIB
```

On each pfSense (use `sh -c` under tcsh):

```
vtysh -c "show ip ospf neighbor"
vtysh -c "show ip route"
netstat -rn -f inet | awk '$1=="default"||$1~/^172|^199|^10/'
```

"The config has the line" is not "the kernel installed the route" — image-baked junk routes, equal-cost statics and self-loop next-hops all make FRR drop routes from the FIB. Confirm the FIB, not the config. On `bs-ops-fw` specifically, confirm the OT umbrella (`172.16.0.0/16`) is a static route that `redistribute_static` has pushed into OSPF.

L2 — pfSense config integrity

```
xmllint --xpath 'string(//gateways/defaultgw4)' /cf/conf/config.xml; echo
pfctl -vsr | grep -c 'label "USER_RULE'
```

L3 — end-to-end from a workstation

```
Test-NetConnection -ComputerName 172.31.12.3 -Port 80    # DMZ web reachable
tracert -d 172.16.46.18                                   # OT historian reachable through the L3.5 boundary
```

```
Resolve-DnsName bs-dc01 -DnsOnly                    # AD DNS only, not LLMNR
ping bs-dc01                                        # resolves to 172.31.2.7, not a mgmt IP
```

Verification philosophy

Gates assert what the next step actually needs, and they must block only on **self-clearing** conditions — a latched flag makes an `until:` unpassable, and `failed_when: false` does not rescue an exhausted `until` (use `ignore_errors`). A gate that passed "last time" proves nothing when the thing it guards is timing-dependent. This is why `so_manager` verifies repos-present and services-up rather than the `so-setup` exit code, and why the `dns` play gates on the `DomainDnsZones` partition rather than assuming it exists.

08 — Operations and Troubleshooting

Credentials

The Security Onion web/join secrets and the OT (OpenPLC / FUXA / InfluxDB / fuel-DB) and Malcolm secrets live in `group_vars/all/vault.yml` (ansible-vault). The OS and device **connection** accounts and the domain-admin account live in the per-group `group_vars`. The values below are the current range **defaults** — rotate them before any deployment that is not a lab.

The two variants are built on **different platform image generations**, which is the one credential difference that bites: the monitoring-only variant runs the 1.1.x images (Windows local admin `simspace`), the Splunk-stacked variant the 1.2.x images (Windows local admin `xadmin`). Using the wrong account for the image generation does not fail fast — it retries to timeout and looks like an unbooted host.

SYSTEM	ACCESS	USERNAME	PASSWORD
Security Onion SOC	`https://172.31.7.15`	`admin@blackstone.mil`	(vault)
SO node join secret	—	—	(vault)
Domain admin (both domains)	`blackstone.mil` / `fops.blackstone.mil`	`simspace` (mon-only) · `xadmin` (stacked)	`Simspace1!Simspace1!` (mon-only) · `ConfigingInTheNameOf1!` (stacked)
Domain users	both domains	seeded identities	(the domain-admin password)
Windows hosts	RDP / WinRM	`simspace` (1.1.x) · `xadmin` (1.2.x)	`Simspace1!Simspace1!` · `ConfigingInTheNameOf1!`
Linux hosts	SSH	`simspace`	`simspace1`
Ubuntu Desktop 25 (stacked)	SSH / console	`xadmin`	`PocketFullofShellCod3!`
pfSense firewalls	SSH / web	`admin`	`simspace1`
VyOS routers	SSH	`vyos`	`Simspace1!`
OpenPLC / FUXA / InfluxDB / fuel audit DB	OT	role accounts	(vault; default `Simspace1!...`)
Splunk (stacked)	web / mgmt	`admin`	`simspace1`
Splunk cluster secret (stacked)	—	—	`Simspace1!ClusterSecret`
Malcolm UI (stacked)	`https://172.31.7.30`	`xadmin`	(vault)

Rotation

```
ansible-vault edit group_vars/all/vault.yml
```

Rotate the vaulted secrets there. The connection and domain-admin accounts are plaintext in the per-group `group_vars`; change them there, and remember the Windows `ansible_become_user` must track `ansible_user`

(runas has no separate become password, so a mismatch fails escalation on every Windows task). Tasks that embed a credential in a shell command carry `no_log` — Ansible echoes `cmd` on failure and `deploy.sh` tees that to `/var/log/playbook_run.log`.

Known behaviours (look like faults, are not)

- **SOC health-check failures fetching detection content.** SOC tries GitHub and the ET rules server every few minutes for rule updates, AI summaries and playbooks. The range has no egress and runs in airgap mode, so these fail continuously. Detection on ingested data is unaffected.
- **Gateway ARP repair messages during deploy.** The `init` role catches and corrects the platform-level gateway-ARP fault (an impostor MAC answers ARP and ICMP for the gateway on some segments, so the gateway pings while nothing routes off-subnet) before it breaks routing. Seeing the repair is good.
- **Emulated-user noise.** Continuous browsing, file and process activity across `[aue]` is intentional. Detections that only work on a quiet network are not detections.
- **A first-attempt host failure.** Usually ordering, not fault — the host depends on state another reaches asynchronously. Attempt 3 is the confirmation (see the build & deploy chapter).
- **The L3.5 boundary permits everything today.** `bs-ops-fw` carries lab "allow any" rules; blocking ACLs are deferred until every subsystem works. A cross-boundary reachability test that succeeds is expected for now.

Common pitfalls

1. **VyOS image bakes self-loop default routes.** FRR refuses a default whose next-hop resolves to a local interface, so `show ip route` has no `S>* 0.0.0.0/0` even though `host_vars` declares one. Fix with `extra_static_routes_remove` per host.
2. `next_hop` **typo: own /30 IP instead of the peer's.** The inline comment naming the device on the other end of each `next_hop` exists because of exactly this.
3. **Windows DDNS registers both adapters** → `ping <host>` resolves to a management IP half the time. The baseline strip play fixes it; `Resolve-DnsName -DnsOnly` is the reliable check.
4. **pfSense** `/cf/conf/config.xml` **permission denied as non-admin.** `ansible_user: admin` is mandatory on pfSense — the other pre-provisioned user can't copy onto the root-owned config file.
5. **pfSense FRR runtime dir missing on boot** (`/var/run/frr/`); the `pfsense_firewall` role pre-creates it before `service frr onestart`.
6. `gretap` **vs** `gre`**.** Zeek silent, Suricata alerting = layer-3 tunnel. Both ends `gretap`.
7. **pfSense NIC ordering.** Management is on the **first** NIC (`vmx0`); a `host_vars` referencing `vmx0` as a data-plane interface is stale.
8. **The** `dns` **role sets no forwarders** — non-Blackstone names don't resolve until the baseline forwarders play runs; a child-DC `DomainDnsZones` partition may lag on a cold build, which the `dns` play gates on.
9. **FUXA ships without the Modbus driver.** `frangoteam/fuxa` has no `modbus-serial` package, so the device never connects until the `fuel_hmi` role installs it into the container — from the Nexus npm group, through the mgmt proxy, with `--strict-ssl=false` for the internal-CA cert.

The authoritative, dated list of overlay workarounds lives in the range's upstream-fixes log — read it before changing network, AD or SO plumbing; half the time the problem is already diagnosed there.

Re-deploying and resetting

```
# Re-run one phase during development
ansible-playbook playbooks/40-manager.yml

# Re-run a subsystem
ansible-playbook fuel_farm_playbook.yml
ansible-playbook atc_playbook.yml

# Targeted recovery (e.g. a single DC + the Fleet gate on the manager)
BOOT_DELAY=0 ansible-playbook site.yml --limit bs-dc01,soc-so-manager

# Full rebuild on an already-up range
BOOT_DELAY=0 ./deploy.sh
```

A `--tags reset` scope (data-only vs also-config) is a design knob — confirm intent before wiring destructive resets.

Re-pinning the blueprint to a new tarball

The deploy chain is driven by the blueprint, not git. After building and pushing a new tarball:

1. Build, commit and push the tarball (see the build & deploy chapter); note the **full** commit SHA.
2. Edit the platform blueprint → the `download-range-vars-script` `content:` → replace the SHA in the `.../<full-sha>/<tarball>` raw URL (`ab_mb.tgz` on the monitoring-only variant, `ab_mbs.tgz` on the stacked variant).
3. Confirm the blueprint references the SHA you pushed.
4. A fresh range fetches that exact tarball via `wget ... -e https_proxy= 10.255.240.1:3128` (curl does **not** work through this proxy on the controller; wget does).

Blueprints are not bundled in the tarball, so this change needs no rebuild — but it only takes effect on a **fresh** range built from the edited blueprint, not on a running one (re-pull the tarball by hand to update a live controller).

Operating gotchas worth internalizing

- **Nexus is reachable only via the mgmt proxy, never directly**, and presents an internal-CA cert. Any new fetcher you add must route through `10.255.240.1:3128` and trust/skip-verify `nexus.dev.ng.simspace.lan`.
- **The vault ships inside the tarball** with its password file — the whole tarball is the trust boundary, so a fresh controller needs no out-of-band vault step, but anyone with the tarball has the vault.
- **Verify the committed tarball, not the working copy** (`git show HEAD:<tarball>`) — a stale or partially-staged tarball has silently shipped before.
- `docs/` **is not deployed** — editing it never changes a running range.
- **An overlay role replaces the base role wholesale**, so a fix to a base role must be a full copy in the overlay, and a fix made in one variant does not propagate to the other — change both when the behaviour should match.